Dopamine in Silicon: Implementing Reward-Modulated STDP in Neuromorphic Hardware

Diana Vins, Gert Cauwenberghs, Leif Gibb

Abstract

Neuromorphic computing platforms offer compelling advantages for energy-efficient inference in spiking neural networks, yet most existing systems rely on offline-trained synaptic weights and lack the capacity for online adaptation. Here, we extend the HiAER-Spike neuromorphic platform to support reward-modulated spike-timing-dependent plasticity (R-STDP), enabling biologically inspired synaptic learning directly within the system architecture. We present two complementary implementations: a software prototype integrated into the HiAER-Spike Python API that serves as an algorithmic reference model for plasticity dynamics, and a hardware extension that moves the full learning pipeline onto the FPGA. The hardware implementation introduces coincidence detection via a double-buffered Active Synapse Buffer, per-neuron eligibility trace storage co-located with membrane potentials in on-chip URAM, and a reward-gated Weight Update Engine that conditionally commits synaptic modifications to High Bandwidth Memory. These components are integrated as incremental extensions to the existing event-driven execution pipeline, preserving the platform's hierarchical address-event routing and scalability. Simulation and testbench experiments confirm correct sequencing of coincidence detection, eligibility accumulation, exponential trace decay, and reward-gated synaptic writeback. Together, these developments transform HiAER-Spike from an inference-oriented architecture into a system capable of supporting on-chip reinforcement learning, providing a foundation for future research in neuromorphic adaptive control, embodied robotics, and computational investigation of dopamine-modulated plasticity in large-scale spiking networks.

Introduction

Modern artificial intelligence has achieved remarkable performance across a wide range of tasks but requires increasingly large computational resources and energy budgets [1], [2], limiting practicality in energy-constrained environments such as mobile platforms, robotics, and edge-computing systems. Neuromorphic computing addresses these challenges by designing systems inspired by biological neural circuits, typically implementing computation through spiking neural networks (SNNs) that communicate via sparse, event-driven spikes rather than continuous numerical operations [3], [4]. This event-driven representation enables highly parallel, energy-efficient processing compared with conventional von Neumann architectures [5]. Large-scale platforms such as IBM's TrueNorth and Intel's Loihi have demonstrated this

approach by implementing millions of spiking neurons while consuming orders of magnitude less power than traditional AI accelerators [6], [7], [8].

HiAER-Spike is a modular neuromorphic platform built around high-performance FPGA hardware that supports massively parallel spike processing, enabling networks of up to approximately 160 million neurons and 40 billion synapses at faster-than-real-time speeds [13]. Despite these capabilities, HiAER-Spike primarily supports event-driven inference rather than on-chip learning: synaptic parameters are trained offline and loaded prior to execution, limiting the system's ability to adapt dynamically to changing environments [13]. Other neuromorphic processors, including Intel's Loihi 2 and mixed-signal systems developed in the European neuromorphic research community, have explored on-chip plasticity through programmable learning engines and three-factor learning rules [7], [15]–[18]; however, these custom-chip approaches typically require specialized hardware pipelines and fixed architectural assumptions [19]–[21]. By integrating plasticity into HiAER-Spike's existing event-driven execution model using reconfigurable FPGA hardware, the approach introduced here preserves scalability and hierarchical routing while enabling rapid algorithmic exploration.

Spike-timing-dependent plasticity (STDP) is a biologically observed learning rule in which synaptic strength is modified according to the relative timing of pre- and postsynaptic spikes [22]–[24]. However, classical STDP cannot directly support reinforcement learning scenarios in which rewards arrive seconds after the neural activity that produced them. Reward-modulated STDP (R-STDP) addresses this distal reward problem by introducing eligibility traces: transient synaptic variables that record spike-timing coincidences and are converted into weight changes only when a global modulatory signal representing reward arrives [25]–[27]. Previous work has demonstrated R-STDP across a range of tasks, including lane-keeping and navigation in robotic systems [28]–[30], visual recognition [31], [32], and hybrid reinforcement learning benchmarks [33], [34], establishing reward-modulated plasticity as a promising mechanism for adaptive neuromorphic intelligence.

In this work, we extend HiAER-Spike to support R-STDP, enabling synaptic learning directly within the system architecture. We first develop a software implementation within the HiAER-Spike Python API to validate the learning rule and provide a reference model, then introduce hardware extensions that integrate coincidence detection, eligibility trace storage and decay, and reward-gated weight updates into the FPGA execution pipeline. Together, these developments transform HiAER-Spike from an inference-oriented platform into one capable of biologically inspired online learning, providing a foundation for future research in neuromorphic reinforcement learning, adaptive robotics, and computational studies of dopamine-modulated plasticity.

Results

Software prototyping establishes reward-modulated synaptic updates in the HiAER-Spike API

We first implemented reward-modulated spike-timing-dependent plasticity (R-STDP) in the HiAER-Spike software stack using an FPGA-in-the-loop workflow. In this setting, network execution uses the existing HiAER-Spike API infrastructure while learning updates are computed in Python between timesteps. To support plastic synapses, we extended the base CRI_network class by introducing a custom CRI_RSTDP_Network class that augments the network's connection representation with explicit synapse identifiers and R-STDP-specific state variables. This implementation also provides a reward-modulated update function that applies weight changes after each timestep according to the current eligibility state and reward signal.

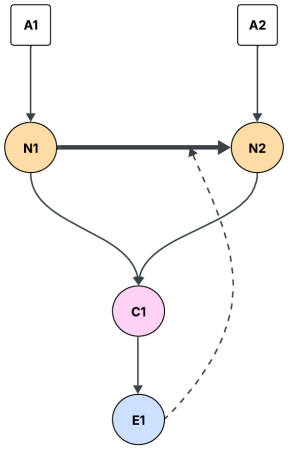

Figure 1. Network motif used to validate hardware R-STDP learning. Axon inputs A1 and A2 drive pre-synaptic neuron N1 and post-synaptic neuron N2, respectively. N1 projects a strong feedforward synapse (bold arrow) to coincidence neuron C1, which also receives input from N2. Each connection to C1 is 0.5 and C1's threshold is 1, so C1 only fires when both pre/post synaptic neurons fire. Neuron E1 receives output from C1 and returns a modulatory feedback signal (dashed arrow) to N2, representing the reward or eligibility pathway. This minimal three-neuron motif isolates the coincidence detection, eligibility trace, and reward-gated weight update mechanisms implemented in the FPGA pipeline.

To demonstrate the functionality of the learning rule, we constructed a minimal network containing [N] neurons and [M] plastic synapses, in which presynaptic and postsynaptic spike timing generates a coincidence signal that drives eligibility accumulation. The network was simulated with a timestep of [1 ms], and spike activity, membrane potentials, eligibility variables, and synaptic weights were recorded throughout the experiment. In this configuration, coincident pre- and postsynaptic activity produced an increase in the eligibility variable associated with the synapse, which subsequently decayed over time in the absence of further activity.

When a reward signal was delivered during the eligibility window, synaptic weights were modified according to the reward-modulated update rule. In representative trials, the synaptic weight increased from w_initial to w_final following reward delivery, whereas identical spike sequences in the absence of reward produced no change in synaptic weight. These results confirm that the software implementation correctly captures the core three-factor learning behavior: spike timing generates eligibility, eligibility persists transiently, and reward gates the conversion of eligibility into synaptic change.

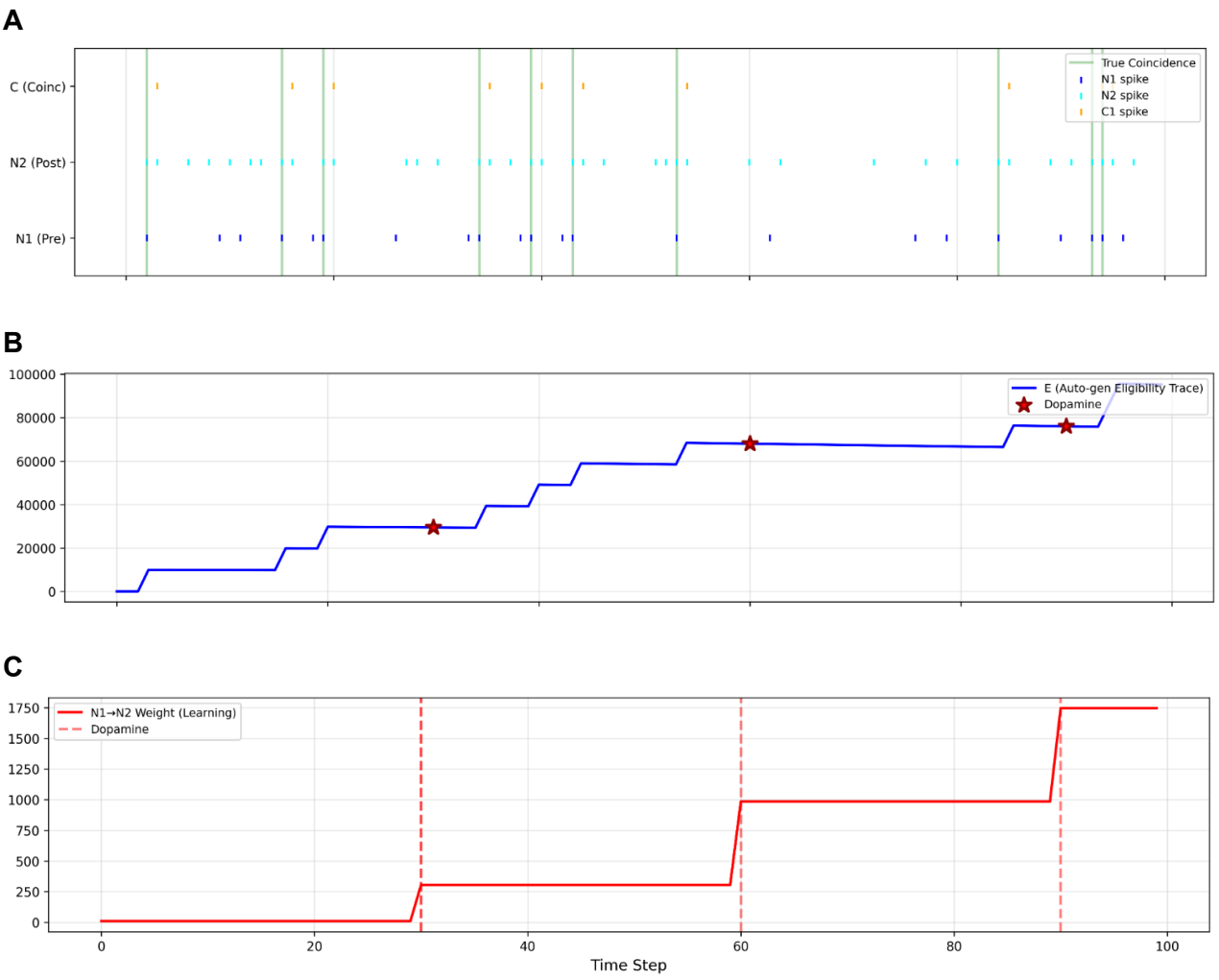

Figure 2. Reward-modulated spike-timing-dependent plasticity demonstrated on a two-neuron circuit. (A) Spike raster showing pre-synaptic neuron N1, post-synaptic neuron N2, and detected coincidence events (C) across 100 timesteps. Vertical green lines mark timesteps in which the hardware coincidence detector identified a co-activation of N1 and N2. **(B)** Eligibility trace accumulated by the FPGA hardware over time. The trace increments at each detected coincidence and decays between events; dopamine reward pulses (★) are delivered at timesteps 28, 60, and 88. **(C)** Synaptic weight of the N1→N2 connection as a function of timestep. Weight updates are committed only at reward delivery, demonstrating that the eligibility trace correctly bridges the temporal gap between synaptic co-activation and reward.

Software demonstrations provide a reference model for hardware implementation

Although the software implementation does not yet incorporate a full reinforcement-learning task, it serves as an algorithmic reference model for the hardware design. The Python implementation exposes the full sequence of operations underlying R-STDP (coincidence

detection, eligibility accumulation, eligibility decay, and reward-gated weight updates) and provides visualization tools for inspecting these internal variables.

Across the prototype experiments, we verified that spike coincidences reliably produced increases in eligibility variables, eligibility values decayed over time in the absence of reward, and reward signals selectively triggered weight updates only when eligibility was present.

Together, these results establish a functional software prototype of R-STDP within the HiAER-Spike programming environment and provide a behavioral reference against which the hardware implementation can be validated.

Hardware extensions enable on-chip reward-modulated synaptic plasticity

After validating the learning rule in software, we implemented the R-STDP pathway directly in the HiAER-Spike hardware architecture. The hardware implementation extends the existing event-driven execution pipeline by introducing mechanisms for coincidence detection, eligibility storage, and reward-gated synaptic writeback.

These capabilities were realized through modifications to two key modules in the HiAER-Spike design. The internal_events_processor module was extended to buffer active synapses, detect spike-timing coincidences, and maintain eligibility traces stored in on-chip memory. The hbm_processor module was extended to support reward-gated weight updates by retrieving eligibility values, computing updated synaptic weights, and writing modified weights back to HBM.

Under this architecture, coincidence events generated during network execution are serialized into a FIFO containing the synapse address, neuron group information, and an update flag. During the learning phase of each timestep, the hardware iterates through these queued events, retrieves the corresponding eligibility value and synaptic weight, computes the updated weight, and commits the modification to memory only when the global reward signal is asserted. This process allows synaptic plasticity to occur entirely on chip without intervention from the host software.

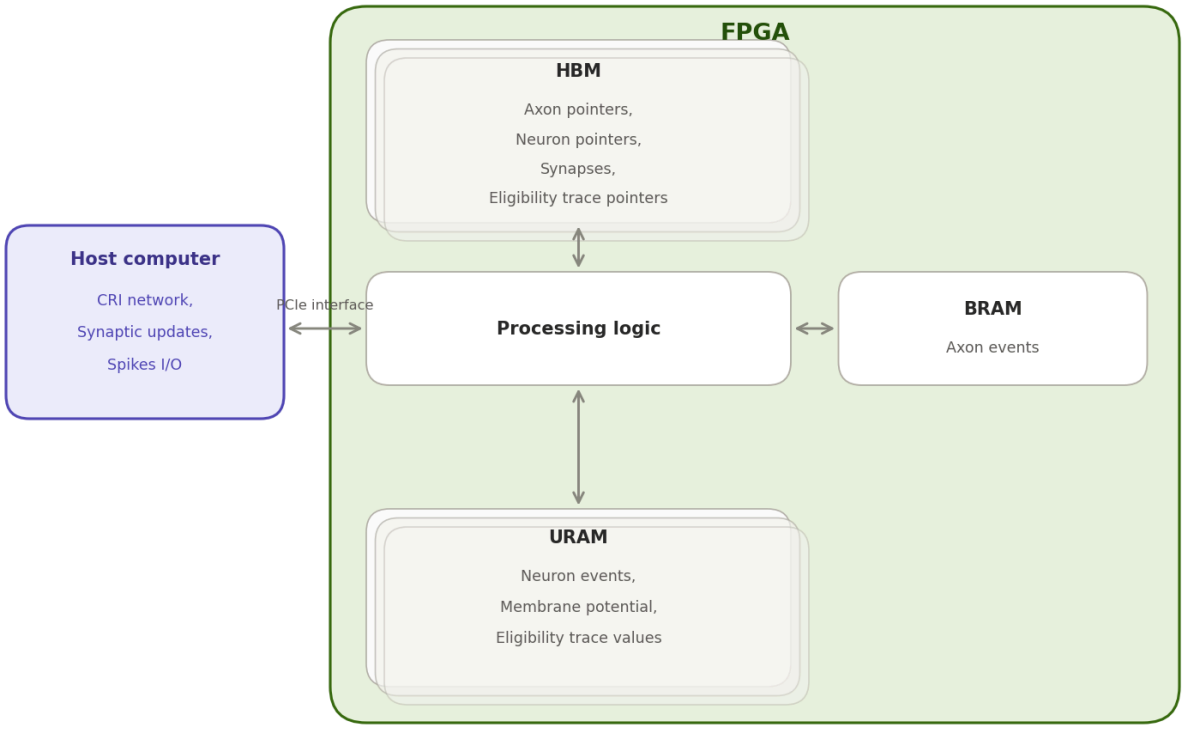

Figure 3. Hardware platform architecture of the FPGA-based neuromorphic accelerator. The system comprises a host CPU communicating via PCIe with a Xilinx UltraScale+ FPGA containing on-chip High Bandwidth Memory (HBM) for synaptic weight and pointer storage, Block RAM (BRAM) for axon events, and Ultra RAM (URAM) banks for membrane potentials and eligibility trace values. Processing logic coordinates all memory traffic and executes the SNN inference and R-STDP learning pipeline.

Hardware verification confirms correct sequencing of the learning pipeline

To verify the functionality of the hardware learning pipeline, we instrumented the design with debug signals reporting the state of weight update events. These signals include the synapse address being updated, the eligibility value used in the update, the original synaptic weight, the computed updated weight, and a write-back validity signal indicating successful memory modification.

In simulation and testbench experiments, the system produced the expected sequence of operations following spike coincidence events. Specifically, a coincidence event generated a queued update request for synapse [addr_example], after which the hardware retrieved an eligibility value of [e_value], computed a new synaptic weight of [w_new] from an initial value of [w_old], and committed the updated value to HBM when the reward signal was asserted.

When reward was disabled, the same read and compute stages occurred but the write-back stage was suppressed, confirming that reward gating controls the final consolidation of synaptic plasticity. This behavior verifies the correct operation of the reward-modulated learning pathway implemented in hardware.

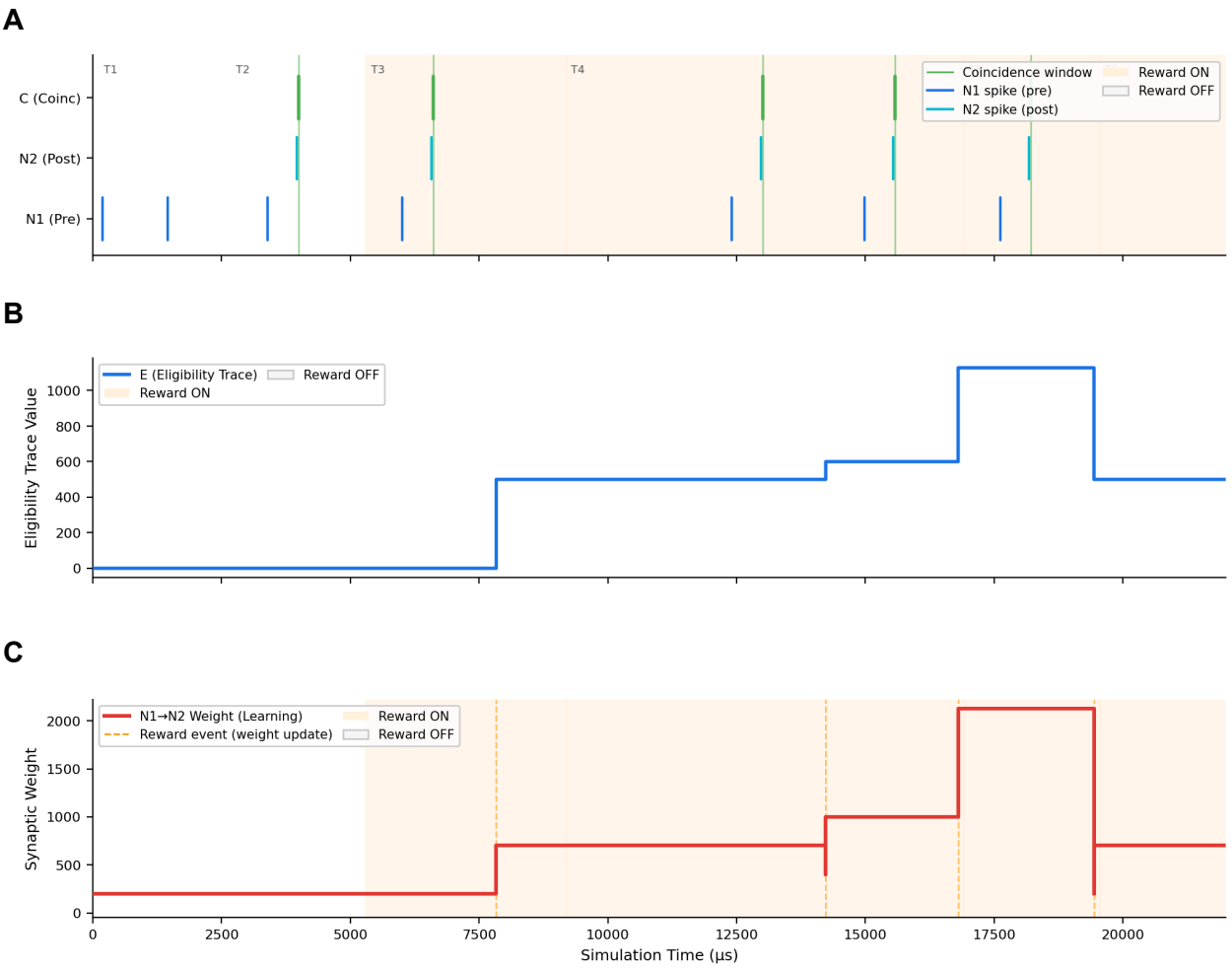

Figure 4. System-level reward-modulated STDP learning in a single-synapse network. Simulation results from the Step 8 integration testbench, covering a representative window spanning Tests 1–5. Orange shading indicates timesteps in which the reward register (exec_reward) is asserted high; unshaded regions indicate reward-off periods. **(A)** Spike raster plot for the pre-synaptic neuron N1 (blue), post-synaptic neuron N2 (cyan), and detected coincidence events C (green). A coincidence is recorded when Phase 1 of the internal events processor detects that N1 was active in the previous timestep and N2 has exceeded the spiking threshold in the current timestep, triggering a push to the coincidence FIFO. **(B)** Eligibility trace (ET) accumulated over time. The ET is updated by a read-modify-write operation on the URAM whenever a coincidence event is processed, adding et_increment to the stored value. Stars mark timesteps in which the weight update engine (WUE) fired (i.e., a coincidence was present and reward was asserted) reading the ET from URAM and the current synaptic weight from HBM to compute the updated weight. When reward is off, coincidences still push to the FIFO but the WUE computation and HBM write-back are suppressed, leaving the ET unchanged at its pre-event value. **(C)** Evolution of the N1→N2 synaptic weight stored in HBM Region 3. The weight increases in discrete steps only when a coincidence event coincides with an asserted reward signal, implementing reward-gated Hebbian plasticity. Dashed vertical lines mark each weight-update event. The weight is held constant across all non-rewarded timesteps, demonstrating correct gating of the write-back path.

Methodology

HiAER-Spike execution model and baseline architecture

All implementations were developed as extensions to the HiAER-Spike hardware–software stack. In the baseline platform, synaptic connectivity is stored in off-chip high-bandwidth memory (HBM), whereas neuron events and membrane state are maintained in on-chip memory resources, including BRAM and URAM. Network execution proceeds in an event-driven manner using hierarchical address-event routing, with sparse pointer-based access to synaptic data and grouped neuron updates. The current public HiAER-Spike description emphasizes scalable inference and large-network execution, with HBM-organized pointer/synapse regions and packetized processing over 16-neuron groups.

To introduce online synaptic plasticity without redefining the full HiAER-Spike architecture, we implemented reward-modulated spike-timing-dependent plasticity (R-STDP) as an incremental extension to the existing execution flow. The work was split into two stages. The first stage used the existing hs_api software interface to emulate R-STDP in an FPGA-in-the-loop setting, with learning logic executed in Python and network execution delegated to the HiAER-Spike software stack. The second stage moved the critical plasticity operations onto the FPGA by augmenting the hardware data path with eligibility-trace storage, coincidence detection, reward-gated update logic, and synaptic writeback support. This division allowed us to establish algorithmic correctness first at the API level and then translate the same learning mechanism into a hardware-oriented realization.

Reward-modulated STDP formulation

We used a pair-based reward-modulated STDP rule derived from prior dopamine-gated STDP formulations, with the update expressed as a reward-scaled eligibility term. In the software prototype, the synaptic weight update followed the form

$$\Delta w = \varepsilon R$$

where ε denotes the eligibility value associated with a synapse, R is the reward or dopamine-like modulatory signal. The software class exposed this explicitly as an apply_rstdp function that computed a weight increment from the current eligibility value and then rewrote the synaptic weight through the existing API.

Although both implementations followed the same overall three-factor principle, the software and hardware realizations were not identical. The software implementation more directly followed the reference R-STDP construction used for prototyping and visualization, whereas the hardware implementation adopted a simplified event-driven realization better matched to FPGA execution. In the hardware version, eligibility and weight updates were tied to spike-timing coincidences detected during runtime, and weight writeback was gated by a global reward register.

Software implementation in hs_api

The software prototype was built by extending the existing CRI_network abstraction in the HiAER-Spike API. Specifically, a child class, CRI_RSTDP_Network, was introduced to support

synapse-specific plasticity metadata while remaining compatible with the original network construction flow. The custom class preserved the same general network specification interface as the parent API but augmented the connection structure so that each synapse could be associated with an explicit identifier and auxiliary R-STDP state. The implementation also added an R-STDP update function to apply reward-gated weight modification programmatically after simulation steps.

In this software formulation, the learning rule was represented by automatically generated auxiliary circuitry for each plastic synapse. For a given pre–post connection, the network constructor created additional internal units corresponding to coincidence-detection and eligibility-trace dynamics. In the two neuron prototype, the class auto-generated coincidence detector (C) and eligibility (E) neurons for each synapse and appended them to the augmented network graph. The class stored a dictionary mapping each plastic synapse to its pre- and postsynaptic units, the associated coincidence unit, the associated eligibility unit, and the initial synaptic weight. The software then used the resulting activity traces to compute eligibility values and reward-modulated weight changes over time.

This implementation used a simplified four-node motif to demonstrate the learning mechanism: a presynaptic neuron, a postsynaptic neuron, a coincidence detector, and an eligibility unit. The network was driven by externally defined inputs, and weight evolution was monitored over repeated timesteps. The implementation employed leaky-integrate-and-fire dynamics for the principal neurons and the eligibility unit, while the coincidence detector used a thresholded artificial-neuron model. Example parameterization in the repository used a high threshold for eligibility units to prevent ordinary spiking, allowing their membrane state to serve as a persistent proxy for eligibility, together with timestep-wise decay to approximate an exponentially decaying trace.

Simulation protocol for software experiments

Software experiments were executed in timestep-based simulation, with one timestep corresponding to 1 ms. Inputs were applied as externally driven spike trains, including stochastic drive generated from Poisson processes in the two neuron demonstration. The implementation recorded membrane potentials, eligibility-like states, synaptic weights, and spike rasters over time. Reward delivery was modeled as a temporally controlled modulatory signal that gated conversion of eligibility into weight change. In the demonstration, dopamine-like events were injected at selected timesteps and the resulting incremental weight updates were visualized directly. This phase of the work was designed not as a full reinforcement-learning task but as a proof-of-mechanism showing that eligibility evolved over time and that synaptic weights changed in the expected direction under reward-gated conditions.

Hardware Platform and System Context

The neuromorphic accelerator is implemented on a Xilinx FPGA with on-chip High Bandwidth Memory (HBM). The execution pipeline is organized around three cooperating RTL modules: the Command Interpreter (CI), the Internal Events Processor (IEP), and the HBM Processor

(HBM). The CI receives configuration commands over a PCIe-backed FIFO from the host CPU and distributes runtime parameters to the other modules. The HBM Processor manages all off-chip memory traffic via an AXI4 read/write interface, issuing burst requests to fetch synaptic weights and spike pointer tables. The IEP maintains 16 parallel URAM banks, one per neuron group (NG), each 72 bits wide and addressed by a 12-bit physical row index; two signed 36-bit values are packed per row, allowing up to 2 × 2^12 = 8192 neurons per group. Membrane potentials are stored in the lower URAM address range; the additions described here extend the upper range to store per-neuron eligibility traces (ETs) in the same physical banks.

The R-STDP learning additions described in this paper extend the existing SNN inference pipeline without modifying its timing or data paths. All learning hardware activates after the standard inference phases of each timestep have completed, then releases control back to the timestep sequencer.

R-STDP Architecture Overview

Reward-modulated spike-timing-dependent plasticity requires four components: a reward signal that gates whether weight updates are committed to memory; a spike coincidence detector that identifies post-synaptic firing the timestep after one of its pre-synaptic neuron fires; an eligibility trace accumulator that sustains a synapse-specific learning signal across time; and a weight update engine that applies the accumulated trace to the stored synaptic weights. Figure 1 maps these four biological components onto the three hardware modules of the accelerator and illustrates the inter-module signal topology.

The reward signal is implemented as a single-bit register, exec_reward, held in the CI and distributed to the HBM Processor. It is set at runtime by the host CPU via the CMD_SET_REWARD command (opcode 0x0A). Within the HBM Processor, this register gates the final AXI4 write-back stage of the Weight Update Engine (WUE): synaptic weights are written to HBM only when exec_reward is asserted. Crucially, when reward is absent the WUE continues to execute its read and compute passes in full, maintaining deterministic pipeline timing and preserving eligibility trace state regardless of reward epoch boundaries.

Rather than implementing a biologically faithful pre-post timing window, the system employs a structural coincidence proxy: a post-synaptic neuron that fires during the threshold evaluation pass of timestep *t* is deemed co-incident with any pre-synaptic input from a synapse packet that was active during the synaptic integration pass of the same timestep. This approximation is computationally efficient and maps naturally onto the existing two-pass execution structure. It is implemented in the IEP through a double-buffered Active Synapse Buffer (ASB) and a fully parallel combinational comparator array, which are described in detail in Sections 4 and 5.

Each neuron maintains a 36-bit signed eligibility trace stored in the upper address range of the same URAM bank that holds its membrane potential, eliminating the need for additional on-chip memory resources. The trace is incremented by a configurable signed value (et_increment) on each detected coincidence event and is decayed by an arithmetic right shift of programmable depth (et_leak_shift) applied once per timestep after all weight updates are complete. The signed 36-bit representation supports both potentiating and depressing traces through the sign of et_increment, and the configurable decay shift implements an exponential leak with time

constants determined by the shift depth. Both parameters are set by the host via the CMD_SET_ET_PARAMS command (opcode 0x0B), and the URAM address range reserved for ET storage is set independently via CMD_SET_ET_RANGE (opcode 0x0C).

The WUE resides within the HBM Processor and activates at the conclusion of Phase 2, draining a coincidence FIFO (CoincFIFO) that is populated by the IEP during the preceding inference passes. For each entry, the WUE fetches the URAM ET pointer for the relevant synapse group from an HBM pointer table in Region 4, requests the current ET value from the IEP via a dedicated two-signal handshake, reads the existing synaptic weight from HBM Region 3, and computes the updated weight as new_w = clamp(old_w + ET, −32768, 32767). The complete signal topology between the three modules is shown in Figure 1; the temporal ordering of all operations within a single timestep is shown in Figure 2.

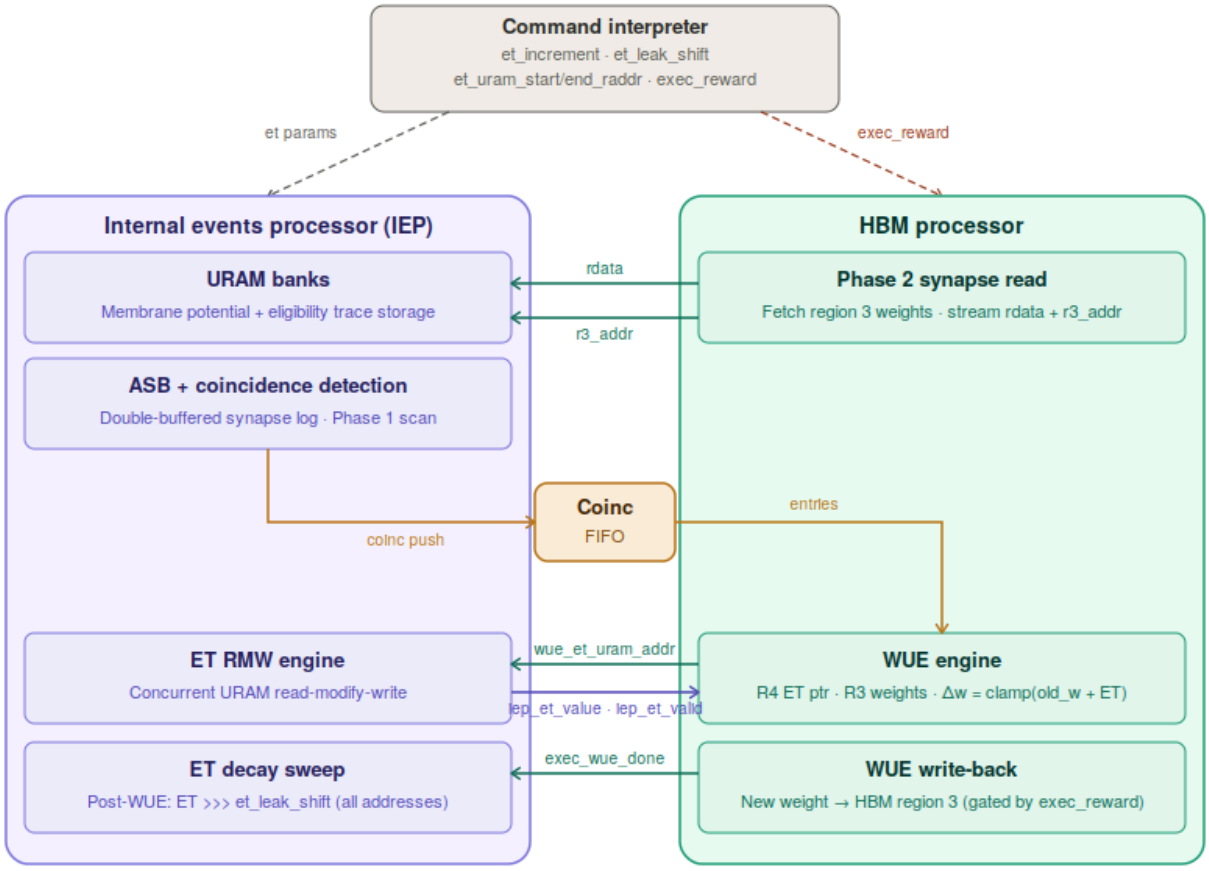

Figure 1. Signal communication topology between the internal events processor (IEP), HBM processor, and command interpreter modules implementing reward-modulated spike-timing-dependent plasticity (R-STDP) on an FPGA neuromorphic accelerator. The command interpreter configures both subsystems at initialisation: eligibility trace (ET) parameters (et_increment, et_leak_shift, et_uram_start/end_raddr) are routed to the IEP, while the reward gate signal (exec_reward) is routed to the HBM processor. During each timestep, the HBM processor streams synaptic weight data (rdata, r3_addr) to the IEP's URAM banks and active synapse buffer (ASB) during Phase 2. The ASB records the HBM region 3 address and the 16 URAM target addresses associated with each packet; at Phase 1, the IEP scans the previous timestep's ASB against current spike activity to detect co-activation, pushing coincidence entries into the CoincFIFO as {do_weight_update, group_mask, r3_addr} tuples. The HBM processor's weight update engine (WUE) drains the CoincFIFO, fetching the ET URAM pointer from HBM region 4 and requesting the corresponding ET value from the IEP's concurrent ET read-modify-write (RMW) engine via a

wue_et_uram_addr / iep_et_valid handshake. The WUE computes the updated synaptic weight as new_w = clamp(old_w + ET, −32768, 32767) and writes it back to HBM region 3, gated by exec_reward. On exec_wue_done, the IEP sweeps all ET URAM addresses and applies an arithmetic right-shift decay of et_leak_shift bits, completing the R-STDP update cycle.

Timestep Execution Structure

Each algorithmic timestep is executed as three sequential, non-overlapping passes, as illustrated in Figure 2. The first two passes constitute the standard SNN inference pipeline; the third is the novel learning pass added by this work.

The first pass, referred to as Phase 1, is the neuronal inference pass. The HBM Processor fetches spike pointer tables that encode the connectivity of active axon groups, and the IEP reads membrane potentials from URAM, accumulates incoming spike-weighted inputs received from the External Events Processor (EEP), and compares each updated potential against the signed 36-bit global threshold. Neurons whose accumulated membrane potential exceeds the threshold emit a spike flag in the 16-bit exec_uram_spiked output mask. Concurrently, the coincidence detection logic within the IEP scans the previous timestep's ASB bank to identify any currently-firing neuron whose URAM address matches a synaptic target address recorded during the preceding Phase 2. Any detected coincidences are immediately serialized into the CoincFIFO as do_weight_update = 1 entries.

The second pass, Phase 2, is the synaptic read and membrane update pass. The HBM Processor issues AXI4 burst reads to Region 3 of HBM, returning 512-bit packets each containing 16 signed 16-bit synaptic weights together with a 23-bit Region 3 base address (exec_hbm_region3addr) that identifies the location of those weights in memory. The IEP sign-extends each 16-bit weight to 36 bits and accumulates it into the corresponding membrane potential in URAM. Simultaneously, for each packet arrival, the IEP writes one entry into the current write-bank of the ASB, recording both the R3 address and the 16 URAM target addresses that were in-flight at that cycle. This recorded state provides the substrate against which the following timestep's Phase 1 coincidence scan will operate. During Phase 2, the IEP also pushes membrane-only entries (do_weight_update = 0) to the CoincFIFO for axon groups that are active but did not produce a Phase 1 coincidence, enabling bidirectional ET maintenance.

The third pass is the WUE learning pass. On completion of Phase 2, the HBM TX finite state machine (FSM) transitions to TX_STATE_WUE_POP_COINCFIFO and begins sequentially draining the CoincFIFO. Each entry triggers a pair of AXI4 read operations: one single-beat read to Region 4 to retrieve the ET URAM pointer, and one two-beat read to Region 3 to retrieve the old synaptic weight. This is followed by an ET read-modify-write operation in the IEP and a conditional weight write-back to HBM. When the CoincFIFO is empty, the HBM Processor asserts exec_wue_done, which the IEP receives as the signal to begin the ET decay sweep. The timestep sequence then resets for the following cycle.

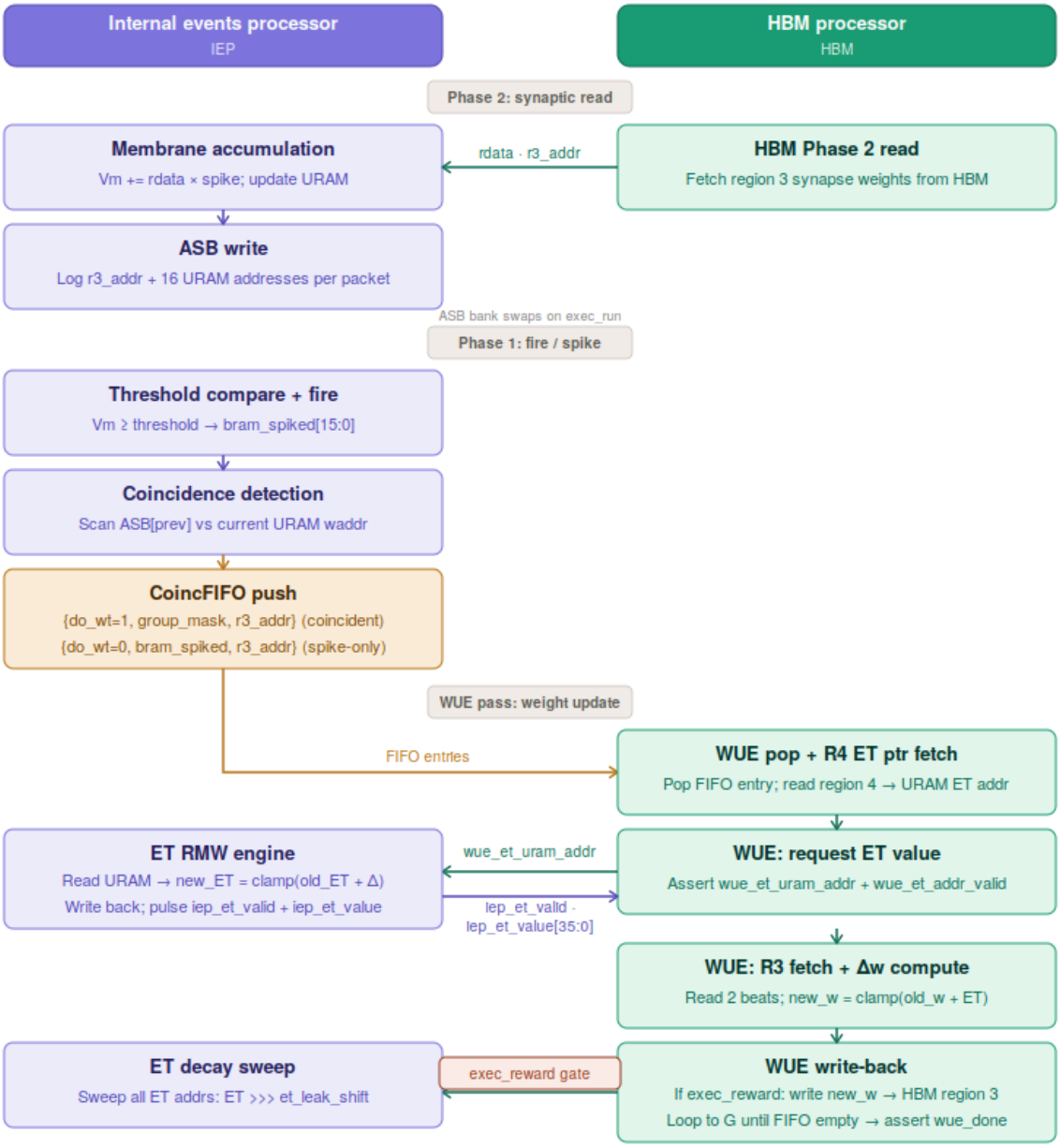

Figure 2. Single-timestep execution sequence for reward-modulated spike-timing-dependent plasticity (R-STDP) across the internal events processor (IEP) and HBM processor modules. Each timestep proceeds through three sequential passes. In Phase 2 (synaptic read), the HBM processor streams synapse weight data and their corresponding HBM region 3 addresses to the IEP, which accumulates weighted inputs onto each neuron's membrane potential stored in URAM. Simultaneously, each received packet is logged into the double-buffered Active Synapse Buffer (ASB), recording both the region 3 address and the 16 URAM target addresses for that packet. In Phase 1 (fire/spike), the IEP compares each neuron's membrane potential against threshold and emits a spike mask. The IEP then scans the previous timestep's ASB bank against the current cycle's URAM write addresses to detect co-activation: synapses whose pre-synaptic inputs were active in the preceding timestep while the post-synaptic neuron fires in the current one. Detected coincidences are pushed to the CoincFIFO as entries carrying a weight-update flag, a per-group spike mask, and the region 3 address; spiking-only events without coincidence are pushed with the weight-update flag cleared for membrane-state tracking. In the WUE pass (weight update), the HBM processor drains the CoincFIFO one entry at a time. For each entry it fetches the eligibility trace (ET) URAM address

from HBM region 4, then requests the corresponding ET value from the IEP's concurrent ET read-modify-write (RMW) engine via a handshake (wue_et_uram_addr / iep_et_valid). The RMW engine reads the stored ET, adds the configured et_increment, saturates to the 36-bit signed range, and returns the result. The HBM processor then fetches the current weight from region 3 and computes the updated weight as new_w = clamp(old_w + ET, −32768..32767). Write-back to HBM is gated by the exec_reward signal: weight changes are committed only when a reward is asserted. Once the CoincFIFO is empty, exec_wue_done is asserted, triggering the IEP to sweep all ET URAM addresses and apply an arithmetic right-shift decay (ET >>> et_leak_shift) before the next timestep begins. ET configuration parameters (et_increment, et_leak_shift, and the URAM sweep range) are supplied by the command interpreter at network setup time.

Phase 2: Synaptic Integration and Active Synapse Buffer

During Phase 2, the HBM Processor issues AXI4 burst read requests using pointer addresses drawn from the pointer FIFO. Each burst returns a 512-bit data packet containing 16 signed 16-bit weights, one per neuron group (NG), along with per-group 13-bit URAM routing addresses encoded in the upper payload bits. The IEP sign-extends each 16-bit weight to 36 bits and accumulates it into the target neuron's membrane potential via the existing URAM read-modify-write pipeline. A hazard-bypass register prevents read-after-write corruption: when successive pipeline stages address the same URAM row, the bypass replaces the stale URAM read data with the most recent computed write data.

To support the coincidence detection that occurs in Phase 1 of the subsequent timestep, the IEP constructs a record of all synapse packets processed during Phase 2 by writing to the Active Synapse Buffer (ASB). The ASB is a double-buffered register array comprising two banks of 64 entries each. Every entry stores the 23-bit HBM Region 3 base address of the originating packet (asb_region3) and, for each of the 16 neuron groups, the 13-bit URAM full address that was being read at the moment the packet was processed (asb_target). A new entry is written on each rising edge of exec_hbm_rvalidready_d1 (a one-cycle delayed version of the AXI valid-ready handshake) provided the IEP is in state STATE_POP_PTR_FIFO, Phase 2 is active, and the 64-entry depth limit has not been reached. At the start of each timestep (exec_run), the bank-select register asb_sel inverts, promoting the just-written bank to the role of the read bank for the upcoming Phase 1 scan and invalidating all entries in the newly designated write bank. The entry count of the promoted bank is preserved as asb_prev_depth to bound the Phase 1 comparator traversal.

In addition to the ASB write, the IEP monitors active axon group flags from the EEP (exec_bram_spiked) during Phase 2. When this signal is nonzero for a given packet, and when no Phase 1 coincidence entry for the same R3 address has already been pushed to the CoincFIFO in the current timestep (coinc_r3_match == 0), the IEP pushes a membrane-only entry to the CoincFIFO with do_weight_update = 0, carrying the active group mask and the R3 address. This secondary path enables the WUE to traverse and maintain ET pointers for neurons that received synaptic input without firing, supporting Hebbian depression as well as potentiation depending on the sign of et_increment. A single-bit per-packet guard register (phase2_packet_push_seen) prevents duplicate entries from being generated within the same packet service window.

Phase 1: Threshold Evaluation, Spiking, and Coincidence Detection

Phase 1 of the IEP proceeds through the pre-existing state sequence STATE_FILL_PIPE_PHASE1 → STATE_PUSH_PTR_FIFO → STATE_PHASE1_DONE. During each clock cycle spent in STATE_PUSH_PTR_FIFO, the IEP reads two membrane potentials from the current URAM row (packed as the upper and lower signed 36-bit halves of a 72-bit word) and compares each against the global threshold in a combinational comparator. The least significant bit of the current write address, uram_waddr[G][0], selects which half of the 72-bit word corresponds to the neuron under evaluation, ensuring the parity alignment between the read pipeline and the threshold comparison is preserved throughout the scan. Neurons whose potential exceeds the threshold assert their corresponding bit in exec_uram_spiked[15:0].

Coincidence detection is performed in parallel with threshold evaluation using a synthesized comparator array instantiated via Verilog generated constructs. For each of the 16 neuron groups G and each of the 64 ASB entries E in the previous-timestep read bank, a single-bit match signal asb_match[G][E] is asserted whenever asb_valid[~asb_sel][E] is true and asb_target[~asb_sel][E][G] equals the current write address uram_waddr[G]. The group-level coincidence flag coinc_detected[G] is the bitwise OR of all 64 match bits for group G, resulting in a fully parallel 64-wide comparison per group evaluated in a single combinational stage. A separate spike detection signal, coinc_spiked[G], re-evaluates threshold crossing for the coincidence path using the same write-address parity convention but operating independently of the registered exec_uram_spiked output; this avoids a one-cycle startup latency that would cause the first scan address to be missed. The per-group Phase 1 coincidence mask is formed as the bitwise AND of these two signals:

$$phase1\_coincidence[G] \;=\; coinc\_spiked[G] \;\&\; coinc\_detected[G]$$

When phase1_coincidence is nonzero, the R3 addresses associated with the matching ASB entries are resolved by a priority loop, asb_matched_r3[G], which scans entries in reverse insertion order so that entry 0 (the earliest-inserted) wins deterministically when multiple ASB entries map to the same post-synaptic neuron. The resolved addresses are latched into coinc_r3_latched on the same clock edge. A push sequencer then drains the pending coincidence mask one bit per clock cycle, writing one 40-bit entry to the CoincFIFO per active group: {1'b1, one_hot_group_mask[15:0], coinc_r3_latched[G][22:0]}. The do_weight_update flag in bit 39 is set to 1, distinguishing these coincidence-driven entries from the membrane-only entries (do_weight_update = 0) pushed during Phase 2. The WUE uses this flag to determine whether to perform a weight modification or to traverse ET pointer structures without modifying weights.

Eligibility Trace Management

Eligibility traces are co-located with membrane potentials in the same 16 URAM banks, occupying a configurable upper address range that begins at et_uram_start_raddr and ends at et_uram_end_raddr, both 13-bit values programmed at initialization via the CMD_SET_ET_RANGE command. Each 72-bit URAM word stores two signed 36-bit ET values using the same upper/lower packing as membrane potentials, allowing the existing read and write infrastructure to be reused without modification. The separation between the membrane potential and ET regions is defined entirely by address range and requires no structural change to the URAM banks themselves.

ET updates are performed by a five-state concurrent read-modify-write sub-FSM, et_rmw_state, which runs independently of the main 15-state IEP FSM and shares the URAM port infrastructure. Its states, ET_RMW_IDLE, ET_RMW_READ, ET_RMW_WAIT, ET_RMW_WRITE, and ET_RMW_DONE, are sequenced as follows. When the WUE asserts wue_et_addr_valid together with the 13-bit target address wue_et_uram_addr and 4-bit bank index wue_et_group_idx, the sub-FSM latches both values and enters a wait condition, holding in ET_RMW_IDLE until the main FSM's shared URAM read-enable bus (uram_rden) is deasserted, thereby avoiding bus contention. It then transitions to ET_RMW_READ, broadcasting et_rmw_addr_latched to all 16 uram_raddr_N_full address buses while asserting the read enable for the target bank only. After one clock cycle of URAM read latency in ET_RMW_WAIT, the sub-FSM enters ET_RMW_WRITE, selects the target bank's output through a 16-input combinational multiplexer et_rmw_rdata_mux, extracts the relevant 36-bit half-word based on address parity, and performs the update:

$$et\_rmw\_sum \;=\; old\_ET \;+\; et\_increment$$

$$new\_ET \quad =\; clamp(et\_rmw\_sum,\; -\,2\text{^}35,\; 2\text{^}35 \;-\; 1)$$

The clamped result is written back through the target bank's write port, and iep_et_valid is asserted for one clock cycle with iep_et_value = new_ET, completing the handshake to the WUE. The sub-FSM then returns to ET_RMW_IDLE. To prevent the main FSM's RMW hazard-bypass logic from consuming stale read data in the cycle immediately following the ET write-back, ET_RMW_WRITE is given the highest priority in the uram_rmwdata combinational bypass multiplexer, overriding the normal write-address comparison logic with raw URAM read data.

Following the completion of the WUE pass, the assertion of exec_wue_done transitions the main IEP FSM from its post-Phase-2 idle condition into a sequential ET decay sweep implemented across three new states: STATE_ET_DECAY_READ (4'd12), STATE_ET_DECAY_WAIT (4'd13), and STATE_ET_DECAY_WRITE (4'd14). The sweep iterates over every URAM address in the configured ET range, reading all 16 banks in parallel on each cycle. Both 36-bit halves of each 72-bit word are subjected to a signed arithmetic right shift by et_leak_shift positions, implementing the exponential decay ET ← ET >>> et_leak_shift, before being written back to the same address. By performing the decay across all 16 banks simultaneously on a word-by-word sweep, all neurons are updated in a number of clock cycles equal to the ET address range, introducing no additional latency beyond the sweep duration itself. Once the sweep address reaches et_uram_end_raddr, the FSM asserts exec_uram_phase2_done and the timestep is declared complete.

Weight Update Engine and Reward Gating

The Weight Update Engine is integrated within the HBM Processor and is activated by the transition of the TX FSM to TX_STATE_WUE_POP_COINCFIFO (4'd12), which follows immediately from the Phase 2 completion state rather than returning to idle. The WUE processes entries from the CoincFIFO sequentially, looping through TX_STATE_WUE_POP_COINCFIFO → TX_STATE_WUE_SEND_R4_READ → TX_STATE_WUE_SEND_R3_READ → TX_STATE_WUE_WAIT_RX until the FIFO is empty. A

mirrored RX FSM sequence, RX_STATE_WUE_WAIT_R4 → RX_STATE_WUE_WAIT_R3_BEAT0 → RX_STATE_WUE_WAIT_R3_BEAT1 → RX_STATE_WUE_COMPUTE → RX_STATE_WUE_DONE, handles the incoming data beats and performs the weight arithmetic.

For each CoincFIFO entry, the 16-bit group mask is first decoded to a 4-bit group index by a combinational priority function, onehot_to_idx, which uses a casez statement to select the lowest set bit. The WUE then issues an AXI4 single-beat read (hbm_arlen = 0) to HBM Region 4 at the address region4_offset + r3_addr, retrieving a 256-bit beat whose lower 13 bits contain the URAM ET pointer wue_r4_ptr for this synapse group. On receipt of this beat in RX_STATE_WUE_WAIT_R4, the RX FSM extracts the pointer, asserts wue_et_addr_valid to the IEP along with wue_et_uram_addr = wue_r4_ptr and wue_et_group_idx, and waits for iep_et_valid. The IEP's ET RMW sub-FSM executes the read-increment-write sequence as described in Section 6 and returns the post-increment ET value on iep_et_value, which the WUE latches into wue_et_received.

Concurrently with the ET handshake, the TX FSM issues a two-beat AXI4 read (hbm_arlen = 1) to Region 3 at r3_addr, fetching both 256-bit halves of the 512-bit synapse row into wue_r3_beat0 and wue_r3_beat1. Because the WUE's Region 3 and Region 4 reads are interleaved with the main Phase 2 burst traffic, a 128-entry internal burst descriptor FIFO (BD FIFO, 27 bits per entry: {ptr_addr[22:0], ptr_burst[3:0]}) records each TX-issued burst address at push time and provides the corresponding address to the RX side at pop time. This allows exec_hbm_region3addr to be correctly reconstructed for every received beat irrespective of whether the beat originated from a Phase 2 pointer burst or a WUE read transaction, and prevents the WUE's additional AXI transactions from corrupting the region address tracking logic used by the IEP ASB.

Once both wue_r3_done and wue_et_done are asserted, with the RX FSM tolerating their arrival in either order, the RX_STATE_WUE_COMPUTE state executes the weight arithmetic combinationally. The old weight is extracted as a signed 16-bit field from the concatenated 512-bit synapse word {wue_r3_beat1, wue_r3_beat0} at the bit position determined by wue_group_idx:

$$wue\_old\_w \;=\; wue\_synapse\_comb[(15 \;-\; group\_idx) \times 32 \;+{:}\; 16]$$

$$wue\_sum \;=\; sign\_extend37(wue\_et\_received) \;+\; sign\_extend37(wue\_old\_w)$$

$$new\_w \;=\; clamp(wue\_sum, \;-\; 32768, \; 32767)$$

The updated weight is spliced back into the appropriate 16-bit slot of the relevant 256-bit beat using a combinational multiplexer: groups 8–15 are packed into beat 0 (at address r3_addr) and groups 0–7 into beat 1 (at r3_addr + 1), reflecting the layout written by the network initialization software.

The WUE then proceeds to the write-back path, traversing TX_STATE_WRITE_HBM_ADDR → TX_STATE_WRITE_HBM_DATA → TX_STATE_WRITE_HBM_RESP. The AXI4 write is issued only if exec_reward is asserted; if it is deasserted, the write states are bypassed entirely and the WUE returns immediately to TX_STATE_WUE_POP_COINCFIFO to process the next entry.

This design ensures that eligibility trace accumulation, CoincFIFO population, and the ET RMW handshake all proceed identically regardless of reward state, so that the learning substrate remains continuously updated and ready for weight commitment as soon as reward is restored. Synaptic plasticity is therefore controlled exclusively by a single registered bit, with no structural change to the learning pipeline. When the CoincFIFO is empty, the TX FSM passes through TX_STATE_PHASE2_DONE and asserts exec_wue_done, which the IEP uses as the trigger for the ET decay sweep described in Section 6, completing the timestep.

Discussion

In this work, we demonstrated that the HiAER-Spike platform can be extended to support reward-modulated spike-timing-dependent plasticity (R-STDP), enabling synaptic updates driven by spike timing and reinforcement signals within the system architecture. Our results show that R-STDP can be implemented at two complementary levels within the HiAER-Spike ecosystem. First, a software prototype implemented through the hs_api interface demonstrates how reward-modulated plasticity can be expressed within the existing programming model. This software implementation provides a flexible environment for exploring plasticity dynamics, visualizing spike activity and eligibility traces, and validating learning rules before hardware deployment. The software experiments confirm that the model reproduces the essential characteristics of reward-modulated synaptic plasticity: spike coincidences generate eligibility traces, eligibility persists across timesteps with exponential decay, and reward signals gate the conversion of eligibility into synaptic weight change.

Building on this prototype, we implemented a hardware extension that moves the core plasticity pathway onto the FPGA. The hardware design introduces mechanisms for coincidence detection, eligibility trace storage and decay, and reward-gated synaptic weight updates integrated into the HiAER-Spike event-processing pipeline. Importantly, these modifications were implemented as extensions to the existing architecture rather than a redesign of the platform. Coincidence events generated during spike processing are recorded and later used to trigger weight update computations, while reward signals determine whether computed updates are committed to memory. This design preserves the event-driven execution model of HiAER-Spike while enabling synaptic plasticity to occur entirely on chip.

The transition from FPGA-in-the-loop learning to hardware-resident plasticity represents an important step toward neuromorphic systems capable of adaptive behavior. In the software implementation, learning updates occur externally and must be applied through host-side control. In contrast, the hardware implementation performs eligibility computation, weight update calculation, and synaptic writeback directly within the FPGA execution pipeline.

Beyond its role as a technical extension of the HiAER-Spike platform, the present work also contributes to the broader effort to implement biologically plausible learning mechanisms in neuromorphic hardware. Reward-modulated STDP is widely studied as a candidate neural mechanism for reinforcement learning in the brain, particularly in the context of dopamine-mediated plasticity in basal ganglia circuits. By incorporating R-STDP into a

large-scale neuromorphic architecture, the system provides a potential experimental platform for investigating such learning mechanisms in a controlled hardware environment. In this sense, the platform may serve not only as an engineering tool for energy-efficient learning systems but also as a computational neuroscience testbed for studying reinforcement learning dynamics in spiking networks.

Although the present results demonstrate the feasibility of on-chip R-STDP, several important directions remain for future work. One immediate extension is the evaluation of the learning mechanism in full reinforcement learning tasks. The current experiments focus on validating the plasticity pathway and demonstrating correct eligibility-based weight updates, but the same infrastructure could support closed-loop learning in tasks such as navigation, robotic control, or event-based sensor processing. Previous studies have shown that R-STDP can enable end-to-end learning in tasks such as lane-keeping vehicles, robotic arm control, and visual object recognition. Integrating similar benchmarks into the HiAER-Spike platform would provide a natural next step for evaluating the practical learning capabilities of the system.

Further work is also needed to characterize the energy and performance implications of the hardware learning pathway. Neuromorphic systems are often motivated by their potential for low-power operation relative to conventional machine learning accelerators. Incorporating plasticity mechanisms introduces additional memory accesses and computation stages that may influence system efficiency. Measuring the power consumption, latency, and throughput of the R-STDP pipeline will therefore be important for understanding the trade-offs between adaptability and energy efficiency in neuromorphic hardware.

In summary, this work demonstrates that reward-modulated spike-timing-dependent plasticity can be integrated into the HiAER-Spike neuromorphic platform through both software and hardware implementations. The software prototype provides a flexible environment for exploring plasticity dynamics and validating learning rules, while the hardware extension enables fully on-chip synaptic updates driven by spike timing and reward signals. Together, these developments transform HiAER-Spike from an inference-oriented neuromorphic architecture into a system capable of supporting biologically inspired online learning, laying the foundation for future research in neuromorphic reinforcement learning and adaptive spiking neural networks.

Code Availability

Code is available at https://github.com/dianavins/BISP_199_RSTDP_For_HiAER_Spike.

References

[1] Patterson, D., Gonzalez, J., Le, Q., Liang, C., Munguia, L., Rothchild, D., So, D., Texier, M., & Dean, J. (2021). Carbon emissions and large neural network training. *arXiv:2104.10350*. *(Under review at / published in Communications of the ACM)*

[2] Strubell, E., Ganesh, A., & McCallum, A. (2019). Energy and policy considerations for deep learning in NLP. *Proceedings of the 57th Annual Meeting of the ACL*.

[3] Roy, K., Jaiswal, A., & Panda, P. (2019). Towards spike-based machine intelligence with neuromorphic computing. *Nature*, 575, 607–617. https://doi.org/10.1038/s41586-019-1677-2

[4] Rathi, N., Chakraborty, I., Kosta, A., Sengupta, A., Ankit, A., Panda, P., & Roy, K. (2023). Exploring neuromorphic computing based on spiking neural networks: Algorithms to hardware. *ACM Computing Surveys*, 55(12), Article 243. https://doi.org/10.1145/3571155

[5] Merolla, P. A., Arthur, J. V., Alvarez-Icaza, R., Cassidy, A. S., Sawada, J., Akopyan, F., … Modha, D. S. (2014). A million spiking-neuron integrated circuit with a scalable communication network and interface. *Science*, 345(6197), 668–673. https://doi.org/10.1126/science.1254642

[6] Merolla, P. A., et al. (2014). A million spiking-neuron integrated circuit with a scalable communication network and interface. *Science*, 345(6197), 668–673.

[7] Davies, M., Srinivasa, N., Lin, T.-H., Chinya, G., Cao, Y., Choday, S. H., … Wang, H. (2018). Loihi: A neuromorphic manycore processor with on-chip learning. *IEEE Micro*, 38(1), 82–99. https://doi.org/10.1109/MM.2018.112130359

[8] Davies, M., Wild, A., Orchard, G., Sandamirskaya, Y., Guerra, G. A. F., Joshi, P., … Risbud, S. R. (2021). Advancing neuromorphic computing with Loihi: A survey of results and outlook. *Proceedings of the IEEE*, 109(5), 911–934.

[9] Zhou, Y., Shi, L., et al. (2023). Computational event-driven vision sensors for in-sensor spiking neural networks. *Nature Electronics*, 6, 870–878. https://doi.org/10.1038/s41928-023-01055-2

[10] Chowdhury, S. S., Rathi, N., & Roy, K. (2025). Neuromorphic computing for robotic vision: Algorithms to hardware advances. *Communications Engineering* (Nature portfolio). https://doi.org/10.1038/s44172-025-00492-5

[11] Schuman, C. D., Kulkarni, S. R., Parsa, M., Mitchell, J. P., Date, P., & Kay, B. (2022). Opportunities for neuromorphic computing algorithms and applications. *Nature Computational Science*, 2, 10–19. https://doi.org/10.1038/s43588-021-00184-y

[12] Exploring spiking neural networks for deep reinforcement learning in robotic tasks. *Scientific Reports*, 14, (2024). https://doi.org/10.1038/s41598-024-77779-8

[13] Frank, G., Hota, G., Wang, K., Uppal, A., Olajide, O., Yoshimoto, K., Gibb, L., Wang, Q., Leugering, J., Deiss, Stephen., Cauwenberghs, G. (2025). HiAER-Spike: Hardware-Software Co-Design for Large-Scale Reconfigurable Event-Driven Neuromorphic Computing. IEEE International Conference on Rebooting Computing (ICRC). https://arxiv.org/abs/2504.03671

[14] Frank, G., Hota, G., Wang, K., Deng, C., Arora K., Vins, D., Uppal, A., Olajide, O., Yoshimoto, K, Wang, Q., Yamaoka, M., Leugering, J, Deiss, S., Gibb, L., Cauwenberghs, G. (2026). HiAER-Spike Software-Hardware Reconfigurable Platform for Event-Driven Neuromorphic Computing at Scale. *npj Unconventional Computing*. https://doi.org/10.48550/arXiv.2602.18072

[15] Demirag, Y., Moro, F., Dalgaty, T., Navarro, G., Frenkel, C., Indiveri, G., Vianello, E., & Payvand, M. (2021). PCM-trace: Scalable synaptic eligibility traces with resistivity drift of phase-change materials. *IEEE International Symposium on Circuits and Systems (ISCAS)*. https://doi.org/10.1109/ISCAS51556.2021.9401446

[16] Frenkel, C., & Indiveri, G. (2022). ReckOn: A 28nm sub-mm² task-agnostic spiking recurrent neural network processor enabling on-chip learning over second-long timescales. *IEEE International Solid-State Circuits Conference (ISSCC)*. https://doi.org/10.1109/ISSCC42614.2022.9731734

[17] Moradi, S., Qiao, N., Stefanini, F., & Indiveri, G. (2018). A scalable multicore architecture with heterogeneous memory structures for dynamic neuromorphic asynchronous processors (DYNAPs). *IEEE Transactions on Biomedical Circuits and Systems*, 12(1), 106–122. https://doi.org/10.1109/TBCAS.2017.2759700

[18] Pehle, C., et al. (2022). The BrainScaleS-2 accelerated neuromorphic system with hybrid plasticity. *Frontiers in Neuroscience*, 16, 795876. https://doi.org/10.3389/fnins.2022.795876

[19] Frenkel, C., Bol, D., & Indiveri, G. (2023). Bottom-up and top-down approaches for the design of neuromorphic processing systems: Tradeoffs and synergies between natural and artificial intelligence. *Proceedings of the IEEE*, 111(6), 623–652. https://doi.org/10.1109/JPROC.2023.3273520

[20] Bauer, J., Eshraghian, J. K., et al. (2024). Neuromorphic intermediate representation: A unified instruction set for interoperable brain-inspired computing. *Nature Communications*, 15, 8122. https://doi.org/10.1038/s41467-024-52259-9

[21] Schuman, C. D., Kulkarni, S. R., Parsa, M., Mitchell, J. P., Date, P., & Kay, B. (2022). Opportunities for neuromorphic computing algorithms and applications. *Nature Computational Science*, 2, 10–19. https://doi.org/10.1038/s43588-021-00184-y

[22] Markram, H., Lübke, J., Frotscher, M., & Sakmann, B. (1997). Regulation of synaptic efficacy by coincidence of postsynaptic APs and EPSPs. *Science*, 275(5297), 213–215. https://doi.org/10.1126/science.275.5297.213

[23] Bi, G. Q., & Poo, M. M. (1998). Synaptic modifications in cultured hippocampal neurons: Dependence on spike timing, synaptic strength, and postsynaptic cell type. *Journal of Neuroscience*, 18(24), 10464–10472. https://doi.org/10.1523/JNEUROSCI.18-24-10464.1998

[24] Feldman, D. E. (2012). The spike-timing dependence of plasticity. *Neuron*, 75(4), 556–571. https://doi.org/10.1016/j.neuron.2012.08.001

[25] Solving the Distal Reward Problem through Linkage of STDP and Dopamine Signaling | Cerebral Cortex | Oxford Academic (Izhekevich 2007)

[26] Frémaux, N., & Gerstner, W. (2016). Neuromodulated spike-timing-dependent plasticity, and theory of three-factor learning rules. *Frontiers in Neural Circuits*, 9, 85. https://doi.org/10.3389/fncir.2015.00085

[27] Gerstner, W., Lehmann, M., Liakoni, V., Corneil, D., & Brea, J. (2018). Eligibility traces and plasticity on behavioral time scales: Experimental support of neoHebbian three-factor learning rules. *Frontiers in Neural Circuits*, 12, 53. https://doi.org/10.3389/fncir.2018.00053

[28] End to End Learning of Spiking Neural Network based on R-STDP for a Lane Keeping Vehicle (Bing et al., 2018)

[29] Bing, Z., Baumann, I., Jiang, Z., Huang, K., Cai, C., & Knoll, A. (2019). Supervised learning in SNN via reward-modulated spike-timing-dependent plasticity for a target reaching vehicle. *Frontiers in Neurorobotics*, 13, 18. https://doi.org/10.3389/fnbot.2019.00018

[30] Lu, J., Wang, Y., & Liu, L. (2021). An autonomous learning mobile robot using biological reward modulate STDP. *Neurocomputing*, 458, 114–123. https://doi.org/10.1016/j.neucom.2021.06.052

[31] Mozafari, M., Kheradpisheh, S. R., Masquelier, T., Nowzari-Dalini, A., & Ganjtabesh, M. (2018). First-spike-based visual categorization using reward-modulated STDP. *IEEE Transactions on Neural Networks and Learning Systems*, 29(12), 6178–6190. https://doi.org/10.1109/TNNLS.2018.2826721

[32] Mozafari, M., Ganjtabesh, M., Nowzari-Dalini, A., Thorpe, S. J., & Masquelier, T. (2019). Bio-inspired digit recognition using reward-modulated spike-timing-dependent plasticity in deep

convolutional networks. *Pattern Recognition*, 94, 87–95. https://doi.org/10.1016/j.patcog.2019.05.015

[33] Akl, M., Ergene, D., Walter, F., & Knoll, A. (2023). Toward robust and scalable deep spiking reinforcement learning. *Frontiers in Neurorobotics*, 16, 1075647. https://doi.org/10.3389/fnbot.2022.1075647

[34] Tang, G., Kumar, N., & Michmizos, K. P. (2020). Reinforcement co-learning of deep and spiking neural networks for energy-efficient mapless navigation with neuromorphic hardware. *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 6090–6097. https://doi.org/10.1109/IROS45743.2020.9340948

Supplementary Materials

The following tables enumerate all new ports and commands added to each module. Existing ports and state machines are unchanged.

Table 1. Command Interpreter: new outputs

Signal	Width	Direction	Function
exec_reward	1	output	Reward enable; gates WUE write-back in HBM
et_increment	36 (signed)	output	Per-coincidence ET increment value
et_leak_shift	4	output	ET decay: arithmetic right-shift depth per timestep
et_uram_start_raddr	13	output	First URAM address of ET storage region
et_uram_end_raddr	13	output	Last URAM address of ET storage region

Table 2. Command Interpreter: new host commands

Opcode	Name	Payload fields	Effect
0x08	CMD_NEURON_TYPE	[69:34] threshold, [71:70] neuron model	Sets threshold and neuron model without overwriting input/output counts

0x0A	CMD_SET_REWARD	[0] reward bit	Writes exec_reward register
0x0B	CMD_SET_ET_PARAMS	[35:0] et_increment, [39:36] et_leak_shift	Configures ET increment and decay shift
0x0C	CMD_SET_ET_RANGE	[12:0] start addr, [25:13] end addr	Sets URAM ET sweep range

Table 3. Internal Events Processor: new ports

Signal	Width	Direction	Function
exec_hbm_rx_phase1_done	1	input	HBM Phase 1 completion; gates ASB writes
exec_hbm_region3addr	23	input	R3 address of current synapse packet
exec_wue_done	1	input	WUE completion; triggers ET decay sweep
exec_wue_r3_valid	1	input	WUE R3 data valid; triggers second URAM read in POP state
exec_bram_spiked	16	input	Active axon groups from EEP; used for membrane-only CoincFIFO push
et_uram_start_raddr	13	input	Start of ET URAM region (from CI)
et_uram_end_raddr	13	input	End of ET URAM region (from CI)
et_leak_shift	4	input	ET decay shift depth (from CI)
et_increment	36 (signed)	input	Per-spike ET increment (from CI)
coincfifo_wren	1	output	CoincFIFO write strobe
coincfifo_din	40	output	{do_weight_update[1], group_mask[15:0], r3_addr[22:0]}
coincfifo_full	1	input	CoincFIFO back-pressure

wue_et_uram_addr	13	input	ET URAM address requested by WUE
wue_et_addr_valid	1	input	Pulse: latch address and begin ET RMW
wue_et_group_idx	4	input	URAM bank index for ET RMW
iep_et_value	36 (signed)	output	Post-increment ET value returned to WUE
iep_et_valid	1	output	Pulse: iep_et_value is valid

Table 4. Internal Events Processor: new internal structures and FSM states

Addition	Description
ASB: asb_region3[2][64]	2 banks × 64 entries, 23-bit R3 address per entry
ASB: asb_target[2][64][16]	2 banks × 64 entries × 16 groups, 13-bit URAM address
ET RMW sub-FSM	5 states: IDLE, READ, WAIT, WRITE, DONE
STATE_ET_DECAY_READ (4'd12)	Issues URAM read for current ET address
STATE_ET_DECAY_WAIT (4'd13)	One-cycle URAM read latency wait
STATE_ET_DECAY_WRITE (4'd14)	Writes decayed ET value; increments sweep address

Table 5. HBM Processor: new ports

Signal	Width	Direction	Function
exec_wue_done	1	output	Asserted when CoincFIFO is fully drained
exec_wue_r3_valid	1	output	Asserted when WUE R3 beat 1 is received
exec_hbm_region3addr	23	output	R3 base address of current synapse burst
coincfifo_empty	1	input	CoincFIFO empty flag

coincfifo_dout	40	input	{do_weight_update, group_mask[15:0], r3_addr[22:0]}
coincfifo_rden	1	output	CoincFIFO read enable
region4_offset	23	input	HBM base address of ET pointer table (Region 4)
exec_reward	1	input	Reward gate; suppresses weight write-back when deasserted
wue_et_uram_addr	13	output	ET URAM address sent to IEP ET RMW
wue_et_addr_valid	1	output	Pulse: address on wue_et_uram_addr is valid
wue_et_group_idx	4	output	Target URAM bank index
iep_et_value	36 (signed)	input	ET value returned by IEP
iep_et_valid	1	input	Pulse: iep_et_value is valid
hbm_rx_curr_state	4	output	RX FSM state export for testbench monitoring

Table 6. HBM Processor: new FSM states

FSM	State	Encoding	Function
TX	TX_STATE_WUE_POP_COINCFIFO	4'd12	Pops one entry from CoincFIFO
TX	TX_STATE_WUE_SEND_R4_READ	4'd13	Issues AXI4 single-beat read to Region 4
TX	TX_STATE_WUE_SEND_R3_READ	4'd14	Issues AXI4 two-beat read to Region 3
TX	TX_STATE_WUE_WAIT_RX	4'd15	Waits for RX compute to complete
RX	RX_STATE_WUE_WAIT_R4	4'd10	Awaits R4 ET pointer beat
RX	RX_STATE_WUE_WAIT_R3_BEAT0	4'd11	Awaits first 256-bit R3 beat
RX	RX_STATE_WUE_WAIT_R3_BEAT1	4'd12	Awaits second 256-bit R3 beat

RX	RX_STATE_WUE_COMPUTE	4'd13	Waits for ET and R3 data; triggers weight arithmetic
RX	RX_STATE_WUE_DONE	4'd14	Asserts exec_wue_r3_valid; initiates write-back if exec_reward